函数指针声明解析笔记

题目要求根据给定代码，回答三个问题：

1. 分析函数指针 `f` 的声明；
2. 补全源代码，使其能正确运行；
3. 给出运行结果。

一、声明 `void(*(*f[3])(const char *p))(int x);` 的含义

从名字 `f` 出发，按"先右后左"逐层剥离：

- `f[3]`：`f` 是含 3 个元素的数组；
- `*f[3]`：数组元素是指针；
- `(*f[3])(const char *p)`：指针指向函数，形参为 `const char*`；
- `*(*f[3])(const char *p)`：该函数返回一个指针；
- `(*(*f[3])(const char *p))(int x)`：返回的指针指向形参为 `int` 的函数；
- 最外层 `void(...)`：最终那个函数返回 `void`。

结论：`f` 是一个数组，元素是函数指针；这些函数接收 `const char*`，并返回类型为 `void(*)(int)` 的函数指针。

等价别名写法：

```
using PF  = void(*)(int);         // 指向 void(int) 的函数指针
using PFG = PF(*)(const char*);   // f 的元素类型
PFG f[3];
```

题目已给出的 `void f_sub(int x)` 正是 `PF`。

二、补全思路

`main` 中 `f[0]("Hello")(2)` 分两步执行：

1. 先调用 `f[0]("Hello")`，应打印 `str = Hello`，并返回一个函数指针；
2. 再用参数 `2` 调用返回的指针，即 `f_sub`，打印 `x = 2`。

因此只缺两块内容：

1. 一个接收 `const char*`、打印 `str = ...`、并返回 `f_sub` 的函数（记为 `f_msg`）；
2. 在 `main` 中为 `f[0]` 赋值：`f[0] = f_msg;`。

原声明 `void(*(*f[3])(const char *p))(int x);` 只是定义，数组元素未初始化，直接调用属于未定义行为（野指针），必须补上赋值语句。注意 `f_msg` 需定义在 `main` 之前，`f_sub` 已在文件上方，可直接返回。

三、完整可编译代码

```cpp
/* 班级 学号 姓名 */
#include <iostream>
using namespace std;

/*****************************************************************************
  函数名称：f_sub
  功    能：打印传入的整型参数 x
  输入参数：x —— 待输出的整数
  返 回 值：无
  说    明：题目给定函数，作为"最终被调用的那一层"
*****************************************************************************/
void f_sub(int x)
{
    cout << "x = " << x << endl;
}

/*****************************************************************************
  函数名称：f_msg
  功    能：打印传入的字符串，并返回 f_sub 的函数指针
  输入参数：p —— 指向字符串常量的指针
  返 回 值：void(*)(int) 类型的函数指针（即 f_sub 的地址）
  说    明：新增函数，充当 f 数组元素所指向的"第一层函数"
*****************************************************************************/
void (*f_msg(const char *p))(int x)
{
    cout << "str = " << p << endl;
    return f_sub;          // 函数名自动退化为 void(*)(int)
}

/*****************************************************************************
  函数名称：main
  功    能：通过 f[0] 演示两段式（返回函数指针再调用）的调用过程
  输入参数：无
  返 回 值：0
  说    明：原声明行与两条调用语句保持不动，仅新增一行赋值
*****************************************************************************/
int main()
{
    void(*(*f[3])(const char *p))(int x);

    f[0] = f_msg;          // 新增：把数组首元素指向 f_msg，消除野指针

    f[0]("Hello")(2);
    f[0]("Tongji")(7);

    return 0;
}
```

四、运行结果

```
str = Hello
x = 2
str = Tongji
x = 7
```

与题目截图完全一致：`str` 与 `x` 的值都由 `main` 传入，其中 `x = ...` 两行确实由 `f_sub` 打印。

五、补充说明

- 若不想新增独立函数，也可用 `using PF / PFG` 重写声明类型，或直接把 `f_msg` 的返回类型写成 `PF`，可读性更好，与题目给定声明等价。
- `f[0] = f_msg;` 不可省略。个别编译器上偶发"看起来没崩"，只是恰好命中了可调用地址，仍属未定义行为，不是正确解。